

Different Types of Checked and Unchecked Exceptions

Aim / Objective

To demonstrate different checked and unchecked exceptions in Java using simple programs and exception handling techniques.

Description

The programs demonstrate common exceptions that occur during arithmetic operations, array access, type conversion, file handling, input reading, null references, number conversion, and string indexing.

System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse.

Software: Java Development Kit (JDK), Java compiler, Java Runtime Environment (JRE), and any Java-supported IDE or terminal.

Programs Included

1. ArithmeticExceptionDemo.java
2. ArrayExceptionDemo.java
3. ClassCastExceptionDemo.java
4. IOExceptionDemo.java
5. InputMismatchDemo.java
6. MultipleExceptionDemo.java
7. NullPointerExceptionDemo.java
8. NumberFormatExceptionDemo.java
9. StringIndexExceptionDemo.java

Program 1: Arithmetic Exception

Aim / Objective

To demonstrate handling of ArithmeticException caused by division by zero.

Description

This program demonstrates the Arithmetic Exception using a simple try-catch block. The exception is caught and a meaningful message is displayed.

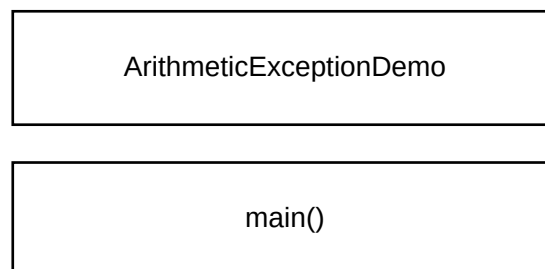
System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse. Software: JDK, Java compiler, JRE, and any Java-supported IDE or terminal.

Algorithm

1. Start the program.
2. Enter the try block.
3. Perform division of 10 by 0.
4. The division causes an ArithmeticException.
5. Catch the exception and display the error message.
6. End the program.

UML Class Diagram



Source Code

```
public class ArithmeticExceptionDemo {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0;  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero.");  
        }  
    }  
}
```

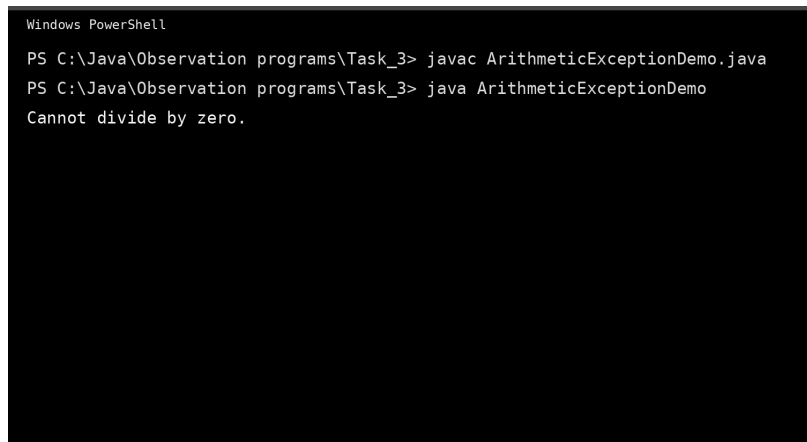
Compilation and Execution Commands

Save as: ArithmeticExceptionDemo.java

Compile: javac ArithmeticExceptionDemo.java

Execute: java ArithmeticExceptionDemo

Output Screen



```
Windows PowerShell  
  
PS C:\Java\Observation programs\Task_3> javac ArithmeticExceptionDemo.java  
PS C:\Java\Observation programs\Task_3> java ArithmeticExceptionDemo  
Cannot divide by zero.
```

Result

Thus, the Arithmetic Exception program was implemented successfully using try-catch exception handling.

Program 2: Array Index Exception

Aim / Objective

To demonstrate handling of an invalid array index using `ArrayIndexOutOfBoundsException`.

Description

This program demonstrates an array index exception by accessing an element outside the valid range and handling it with try-catch.

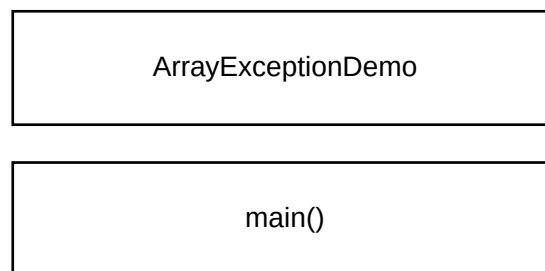
System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse. Software: JDK, Java compiler, JRE, and any Java-supported IDE or terminal.

Algorithm

1. Start the program.
2. Create an integer array with three elements.
3. Try to access index 5.
4. The invalid access causes `ArrayIndexOutOfBoundsException`.
5. Catch the exception and display the error message.
6. End the program.

UML Class Diagram



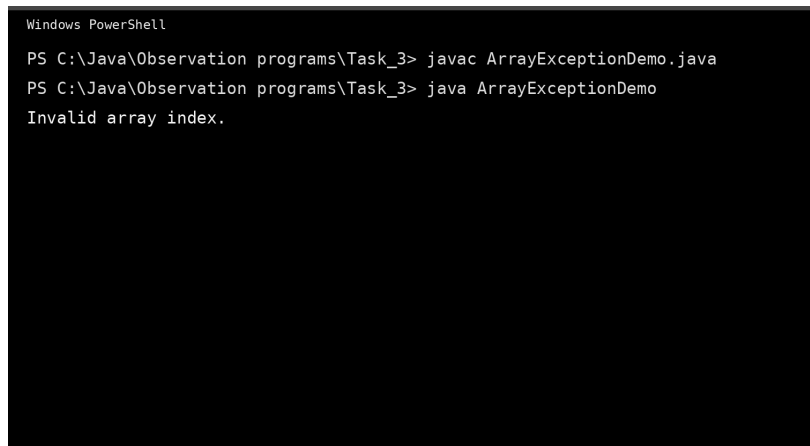
Source Code

```
public class ArrayExceptionDemo {  
    public static void main(String[] args) {  
        try {  
            int[] numbers = {10, 20, 30};  
            System.out.println(numbers[5]);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Invalid array index.");  
        }  
    }  
}
```

Compilation and Execution Commands

Save as: ArrayExceptionDemo.java
Compile: javac ArrayExceptionDemo.java
Execute: java ArrayExceptionDemo

Output Screen



```
Windows PowerShell  
  
PS C:\Java\Observation programs\Task_3> javac ArrayExceptionDemo.java  
PS C:\Java\Observation programs\Task_3> java ArrayExceptionDemo  
Invalid array index.
```

Result

Thus, the Array Index Exception program was implemented successfully using try-catch exception handling.

Program 3: Class Cast Exception

Aim / Objective

To demonstrate handling of ClassCastException during an invalid type conversion.

Description

This program demonstrates an invalid downcast between related classes and handles the resulting ClassCastException.

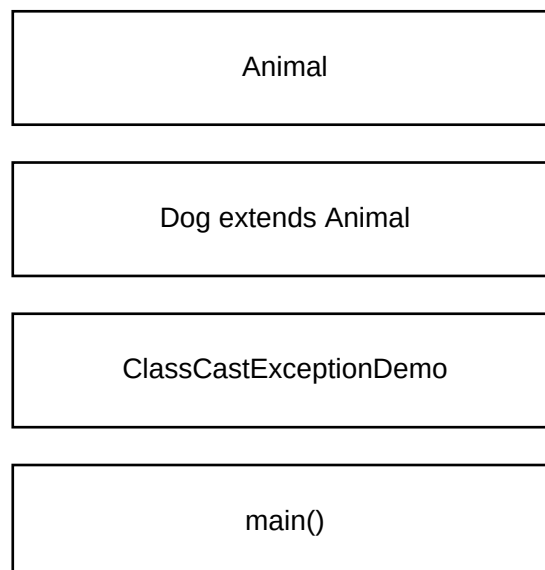
System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse. Software: JDK, Java compiler, JRE, and any Java-supported IDE or terminal.

Algorithm

1. Start the program.
2. Create an Animal object.
3. Try to cast the Animal object to Dog.
4. The invalid cast causes ClassCastException.
5. Catch the exception and display the error message.
6. End the program.

UML Class Diagram



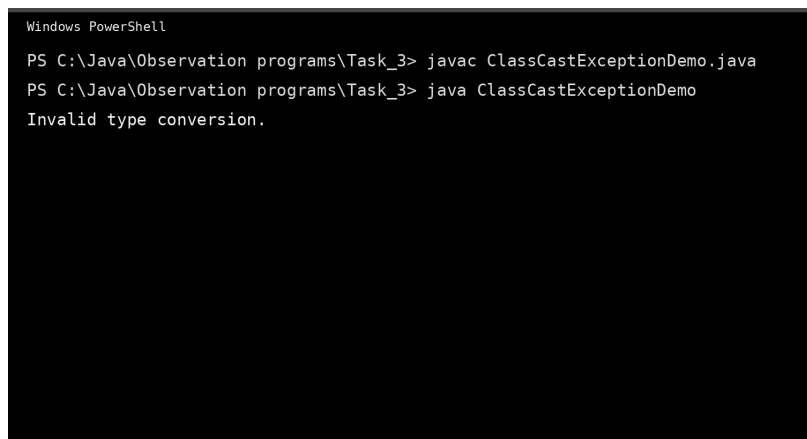
Source Code

```
class Animal {  
}  
  
class Dog extends Animal {  
}  
  
public class ClassCastExceptionDemo {  
    public static void main(String[] args) {  
        try {  
            Animal animal = new Animal();  
            Dog dog = (Dog) animal;  
            System.out.println(dog);  
        } catch (ClassCastException e) {  
            System.out.println("Invalid type conversion.");  
        }  
    }  
}
```

Compilation and Execution Commands

Save as: ClassCastExceptionDemo.java
Compile: javac ClassCastExceptionDemo.java
Execute: java ClassCastExceptionDemo

Output Screen



```
Windows PowerShell  
  
PS C:\Java\Observation programs\Task_3> javac ClassCastExceptionDemo.java  
PS C:\Java\Observation programs\Task_3> java ClassCastExceptionDemo  
Invalid type conversion.
```

Result

Thus, the Class Cast Exception program was implemented successfully using try-catch exception handling.

Program 4: IOException

Aim / Objective

To demonstrate handling of IOException during file access.

Description

This program demonstrates an IOException using FileReader. The exception is caught when the requested file cannot be opened.

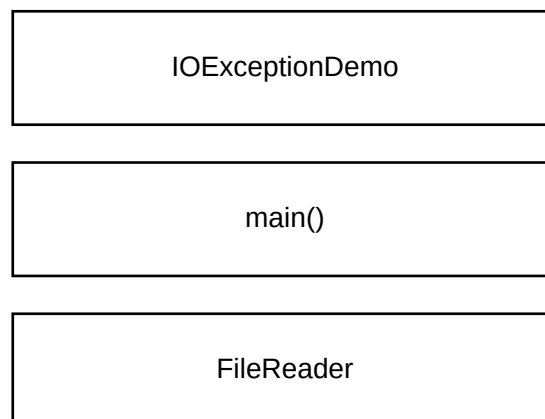
System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse. Software: JDK, Java compiler, JRE, and any Java-supported IDE or terminal.

Algorithm

1. Start the program.
2. Try to create a FileReader for data.txt.
3. If the file cannot be opened, an IOException occurs.
4. Catch the IOException.
5. Display the error message.
6. End the program.

UML Class Diagram



Source Code

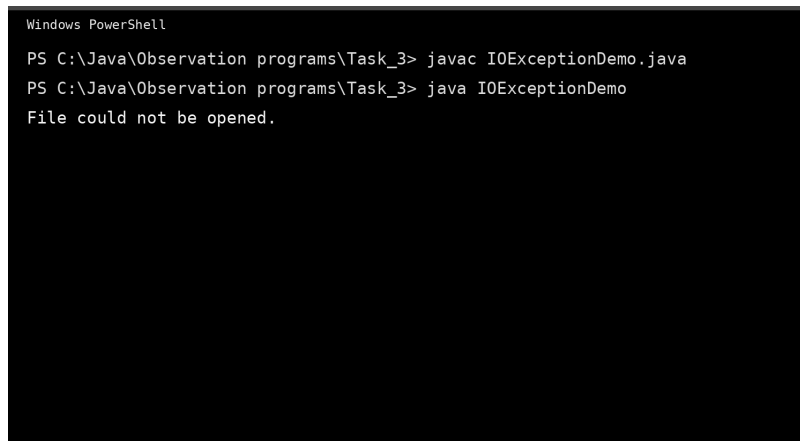
```
import java.io.FileReader;
import java.io.IOException;

public class IOExceptionDemo {
    public static void main(String[] args) {
        try {
            FileReader reader = new FileReader("data.txt");
            reader.close();
        } catch (IOException e) {
            System.out.println("File could not be opened.");
        }
    }
}
```

Compilation and Execution Commands

Save as: IOExceptionDemo.java
Compile: javac IOExceptionDemo.java
Execute: java IOExceptionDemo

Output Screen

A screenshot of a Windows PowerShell terminal window. The title bar at the top reads "Windows PowerShell". The terminal shows the following commands and output:
PS C:\Java\Observation programs\Task_3> javac IOExceptionDemo.java
PS C:\Java\Observation programs\Task_3> java IOExceptionDemo
File could not be opened.
The terminal background is black, and the text is white.

Result

Thus, the IOException program was implemented successfully using try-catch exception handling.

Program 5: Input Mismatch Exception

Aim / Objective

To demonstrate handling of InputMismatchException when the user enters an invalid input type.

Description

This program reads an integer using Scanner. If the user enters a non-integer value, InputMismatchException is caught.

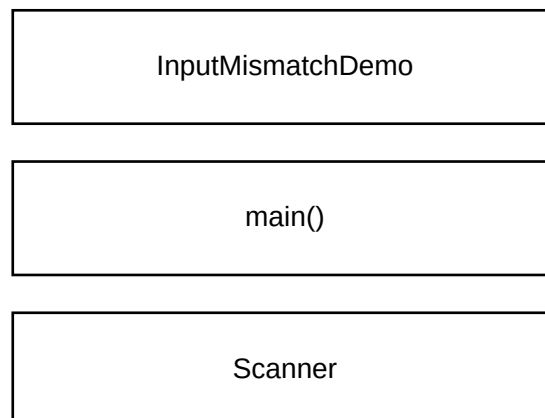
System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse. Software: JDK, Java compiler, JRE, and any Java-supported IDE or terminal.

Algorithm

1. Start the program.
2. Create a Scanner object.
3. Ask the user to enter an integer.
4. Read the value using nextInt().
5. If the input is not an integer, catch InputMismatchException.
6. Display the appropriate message and end the program.

UML Class Diagram



Source Code

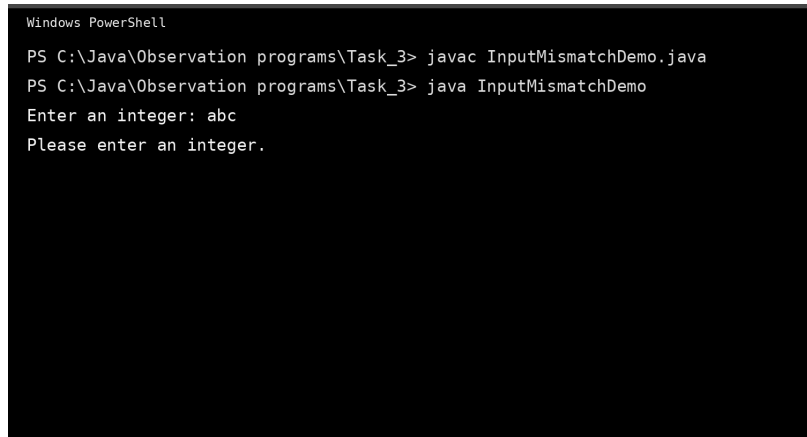
```
import java.util.InputMismatchException;
import java.util.Scanner;

public class InputMismatchDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter an integer: ");
            int number = sc.nextInt();
            System.out.println("You entered: " + number);
        } catch (InputMismatchException e) {
            System.out.println("Please enter an integer.");
        }
        sc.close();
    }
}
```

Compilation and Execution Commands

Save as: InputMismatchDemo.java
Compile: javac InputMismatchDemo.java
Execute: java InputMismatchDemo

Output Screen



```
Windows PowerShell
PS C:\Java\Observation programs\Task_3> javac InputMismatchDemo.java
PS C:\Java\Observation programs\Task_3> java InputMismatchDemo
Enter an integer: abc
Please enter an integer.
```

Result

Thus, the Input Mismatch Exception program was implemented successfully using try-catch exception handling.

Program 6: Multiple Exception Handling

Aim / Objective

To demonstrate handling of more than one exception using separate catch blocks.

Description

This program demonstrates multiple catch blocks. The invalid array access is handled by the appropriate catch block.

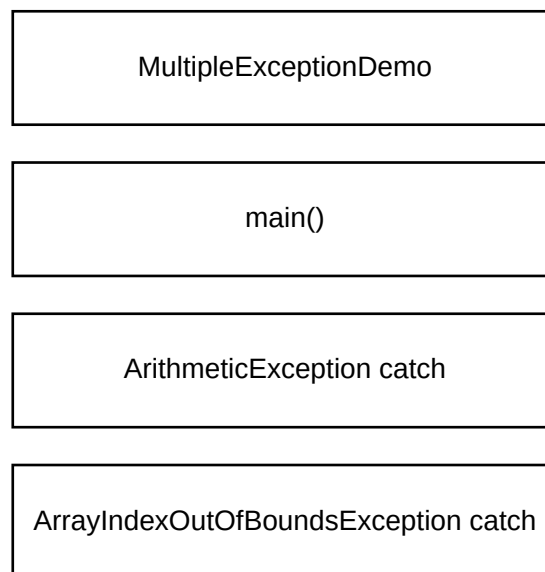
System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse. Software: JDK, Java compiler, JRE, and any Java-supported IDE or terminal.

Algorithm

1. Start the program.
2. Create an integer array.
3. Try to access an invalid array index.
4. Check the exception against the available catch blocks.
5. Catch `ArrayIndexOutOfBoundsException` and display the message.
6. End the program.

UML Class Diagram



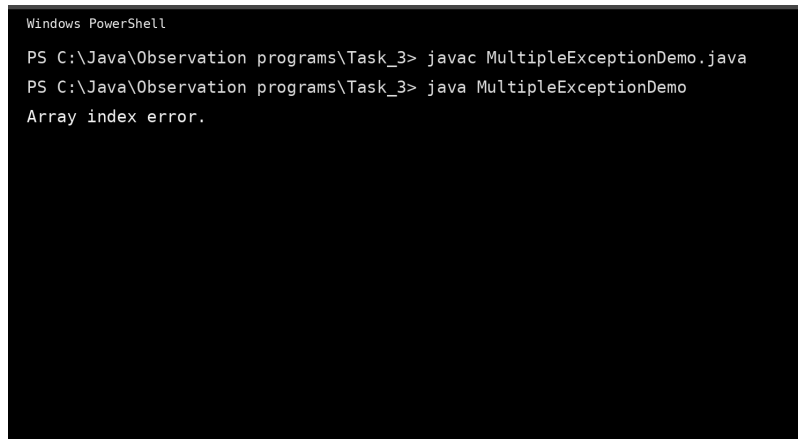
Source Code

```
public class MultipleExceptionDemo {  
    public static void main(String[] args) {  
        try {  
            int[] numbers = {10, 20, 30};  
            System.out.println(numbers[5]);  
        } catch (ArithmeticException e) {  
            System.out.println("Arithmetic error.");  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array index error.");  
        }  
    }  
}
```

Compilation and Execution Commands

Save as: MultipleExceptionDemo.java
Compile: javac MultipleExceptionDemo.java
Execute: java MultipleExceptionDemo

Output Screen



```
Windows PowerShell  
  
PS C:\Java\Observation programs\Task_3> javac MultipleExceptionDemo.java  
PS C:\Java\Observation programs\Task_3> java MultipleExceptionDemo  
Array index error.
```

Result

Thus, the Multiple Exception Handling program was implemented successfully using try-catch exception handling.

Program 7: Null Pointer Exception

Aim / Objective

To demonstrate handling of NullPointerException when a null reference is used.

Description

This program assigns null to a String reference and attempts to access its length. The exception is handled using try-catch.

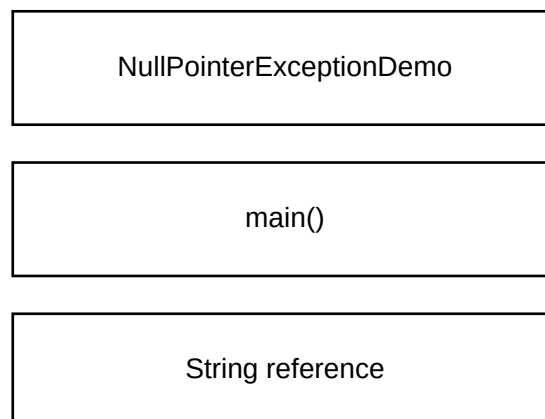
System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse. Software: JDK, Java compiler, JRE, and any Java-supported IDE or terminal.

Algorithm

1. Start the program.
2. Create a String reference and assign null to it.
3. Try to access the length of the String.
4. The operation causes NullPointerException.
5. Catch the exception and display the error message.
6. End the program.

UML Class Diagram



Source Code

```
public class NullPointerExceptionDemo {  
    public static void main(String[] args) {  
        try {  
            String text = null;  
            System.out.println(text.length());  
        } catch (NullPointerException e) {  
            System.out.println("Object reference is null.");  
        }  
    }  
}
```

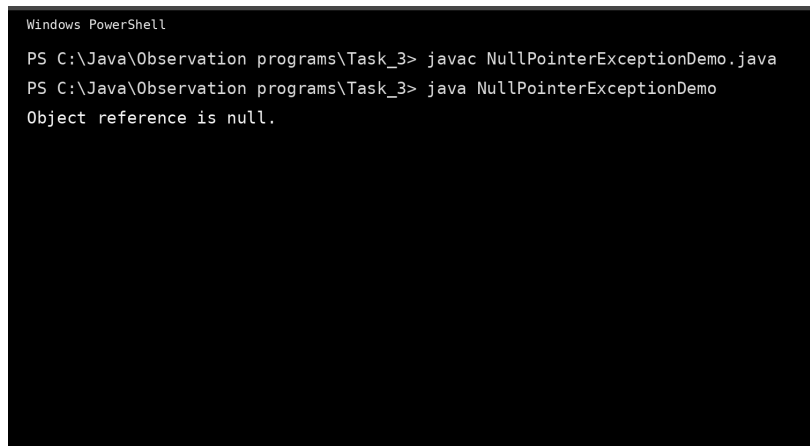
Compilation and Execution Commands

Save as: NullPointerExceptionDemo.java

Compile: javac NullPointerExceptionDemo.java

Execute: java NullPointerExceptionDemo

Output Screen



```
Windows PowerShell  
  
PS C:\Java\Observation programs\Task_3> javac NullPointerExceptionDemo.java  
PS C:\Java\Observation programs\Task_3> java NullPointerExceptionDemo  
Object reference is null.
```

Result

Thus, the Null Pointer Exception program was implemented successfully using try-catch exception handling.

Program 8: Number Format Exception

Aim / Objective

To demonstrate handling of `NumberFormatException` during conversion of an invalid numeric String.

Description

This program attempts to convert a non-numeric String into an integer and handles the resulting `NumberFormatException`.

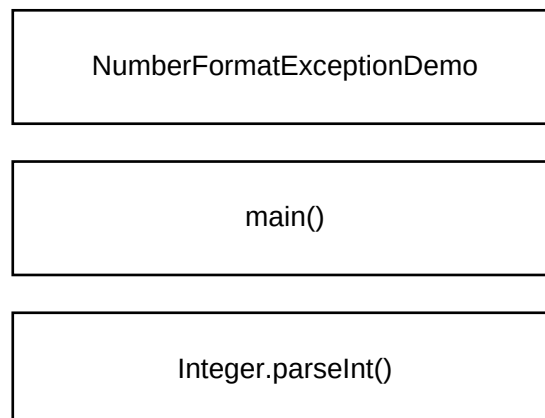
System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse. Software: JDK, Java compiler, JRE, and any Java-supported IDE or terminal.

Algorithm

1. Start the program.
2. Create a String containing a non-numeric value.
3. Try to convert the String to an integer using `Integer.parseInt()`.
4. The invalid conversion causes `NumberFormatException`.
5. Catch the exception and display the error message.
6. End the program.

UML Class Diagram



Source Code

```
public class NumberFormatExceptionDemo {  
    public static void main(String[] args) {  
        try {  
            String value = "abc";  
            int number = Integer.parseInt(value);  
            System.out.println("Number: " + number);  
        } catch (NumberFormatException e) {  
            System.out.println("Invalid number format.");  
        }  
    }  
}
```

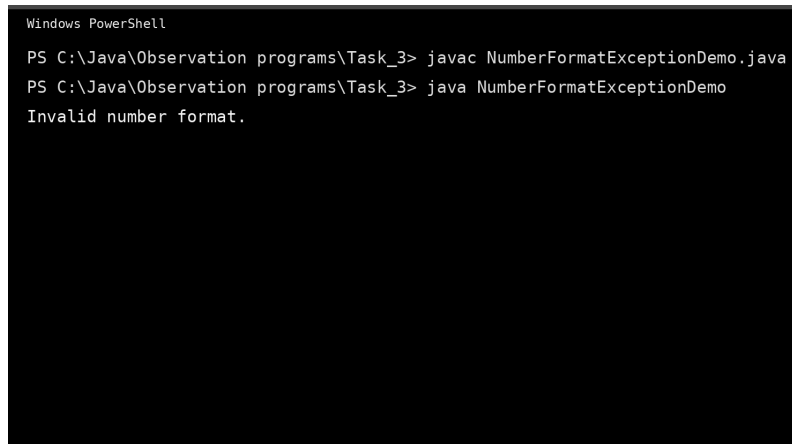
Compilation and Execution Commands

Save as: NumberFormatExceptionDemo.java

Compile: javac NumberFormatExceptionDemo.java

Execute: java NumberFormatExceptionDemo

Output Screen



```
Windows PowerShell  
  
PS C:\Java\Observation programs\Task_3> javac NumberFormatExceptionDemo.java  
PS C:\Java\Observation programs\Task_3> java NumberFormatExceptionDemo  
Invalid number format.
```

Result

Thus, the Number Format Exception program was implemented successfully using try-catch exception handling.

Program 9: String Index Exception

Aim / Objective

To demonstrate handling of `StringIndexOutOfBoundsException` when an invalid string index is accessed.

Description

This program accesses a character outside the valid index range of a `String` and handles the exception using try-catch.

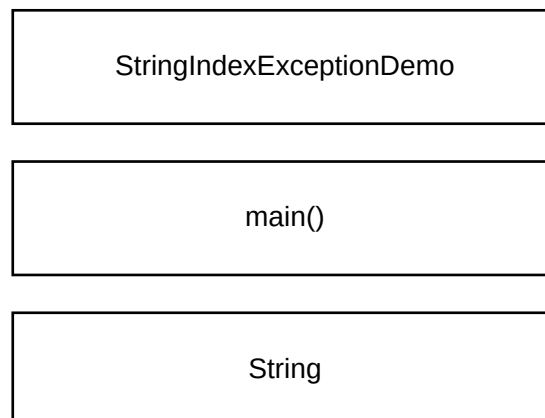
System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse. Software: JDK, Java compiler, JRE, and any Java-supported IDE or terminal.

Algorithm

1. Start the program.
2. Create the `String` "Java".
3. Try to access character at index 5.
4. The invalid index causes `StringIndexOutOfBoundsException`.
5. Catch the exception and display the error message.
6. End the program.

UML Class Diagram



Source Code

```
public class StringIndexExceptionDemo {  
    public static void main(String[] args) {  
        try {  
            String name = "Java";  
            System.out.println(name.charAt(5));  
        } catch (StringIndexOutOfBoundsException e) {  
            System.out.println("Invalid string index.");  
        }  
    }  
}
```

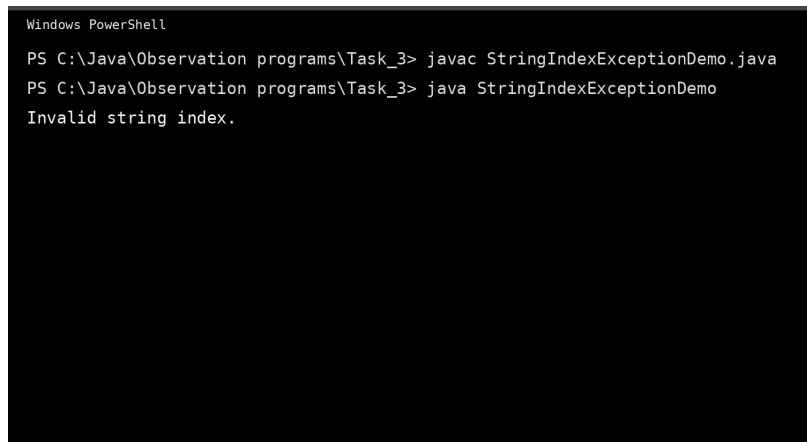
Compilation and Execution Commands

Save as: StringIndexExceptionDemo.java

Compile: javac StringIndexExceptionDemo.java

Execute: java StringIndexExceptionDemo

Output Screen



```
Windows PowerShell  
  
PS C:\Java\Observation programs\Task_3> javac StringIndexExceptionDemo.java  
PS C:\Java\Observation programs\Task_3> java StringIndexExceptionDemo  
Invalid string index.
```

Result

Thus, the String Index Exception program was implemented successfully using try-catch exception handling.